

MilkShow

Documentacao Tecnica do Sistema

Versao:	1.0
Data:	Maio de 2026
Autor:	Frederico Martins
Repositorio:	github.com/fredmartins12/MilkShow

MilkShow -- Documentacao Tecnica do Sistema

Versao: 1.0

Data: Maio de 2026

Autor: Frederico Martins

Repositorio: github.com/fredmartins12/MilkShow

Sumario

1. Descricao Geral do Sistema
2. Estudo de Viabilidade
3. Historias de Usuario
4. Requisitos Funcionais
5. Requisitos Nao Funcionais
6. Casos de Uso
7. Descricao de Casos de Uso
8. Arquitetura de Alto Nivel
9. Diagrama de Classes
10. Diagrama de Sequencias
11. Diagrama de Estados
12. Plano de Testes
13. Telas do Sistema

1. Descricao Geral do Sistema

1.1 Identificacao do Sistema

Nome: MilkShow

Tipo: Sistema de gestao integrada para fazendas leiteiras

Modalidade: SaaS multi-tenant com acesso web, mobile (PWA) e por WhatsApp

1.2 Objetivo

O MilkShow e um sistema de gestao completa voltado para pequenas e medias propriedades leiteiras. Seu objetivo central e digitalizar e centralizar o controle produtivo, sanitario, reprodutivo e financeiro da fazenda, permitindo que o produtor tome decisoes baseadas em dados reais. A grande diferenca do MilkShow em relacao a planilhas e sistemas convencionais e a possibilidade de registrar eventos diretamente pelo WhatsApp usando linguagem natural, sem necessidade de abrir qualquer aplicativo.

1.3 Escopo

O sistema abrange:

- * Gestao do rebanho: cadastro de animais, acompanhamento de status produtivo e reprodutivo.
- * Controle de ordenha: registros individuais e em lote, analise de producao por animal e por periodo.
- * Saude e sanidade: vacinacoes, tratamentos, vermifugacoes e protocolos sanitarios recorrentes.

- * Gestao reprodutiva: inseminacoes, diagnosticos de prenhez, partos e secagem.
- * Bercario: acompanhamento de bezerros, colostro, aleitamento e desmame.
- * Financeiro: receitas, despesas, vendas de leite e animais, custo por litro e rentabilidade.
- * Armazem: controle de estoque de insumos com custo medio ponderado.
- * Inteligencia de negocios (BI): dashboards, graficos, rankings, curva de lactacao, previsao de producao.
- * Bot WhatsApp com IA: registro de eventos por voz, texto e foto, consultas sobre a fazenda em linguagem natural.
- * Painel Mobile: acesso simplificado via PWA React para uso em campo.
- * Protocolos Sanitarios: agendamento e acompanhamento de procedimentos periodicos.

1.4 Usuarios do Sistema

Perfil	Descricao	Forma de Acesso
Produtor Rural	Proprietario da fazenda. Registra eventos e consulta informacoes no dia a dia. Pouco familiarizado com tecnologia.	WhatsApp (principal), App Mobile
Gestor / Tecnico	Responsavel pela analise gerencial da fazenda. Acessa relatorios, BI e configuracoes.	Web (Streamlit)
Administrador do Sistema	Cadastra fazendas, associa numeros de telefone, gerencia multi-tenant.	Web (pagina Configuracoes)

1.5 Contexto de Uso

O produtor rural, durante sua rotina diaria, envia mensagens de voz ou texto pelo WhatsApp informando eventos como ordenhas, compras, gastos e reproducoes. O sistema interpreta essas mensagens com Inteligencia Artificial e persiste os dados no Firebase. O gestor acessa o painel web para analise dos indicadores, relatorios e tomada de decisao.

2. Estudo de Viabilidade

2.1 Viabilidade Tecnica

O sistema e tecnicamente viavel dado que:

- * Python e linguagem madura com vasto ecossistema de bibliotecas para ciencia de dados, ML e web.
- * Streamlit permite criar dashboards interativos de dados com codigo Python minimo, sem frontend dedicado.
- * Firebase Firestore oferece banco de dados NoSQL escalavel, com SDK Python e sincronizacao em tempo real.
- * Evolution API (auto-hospedada) e cascata com Z-API e Twilio garantem integracao confiavel com WhatsApp.
- * LLMs de terceiros (Groq, Gemini, Claude Anthropic) proveem NLP de alta qualidade sem necessidade de treinamento proprio.
- * A infraestrutura de producao e um VPS Hetzner (Ubuntu, R\$50-100/mes), suficiente para dezenas de fazendas.

Principais riscos tecnicos:

Risco	Probabilidade	Impacto	Mitigacao
Instabilidade da API do WhatsApp	Media	Alto	Cascata: Evolution -> Z-API ->
Limites de rate do LLM gratuito	Media	Medio	Cascata: Groq -> Gemini -> Claude (pago)

Perda de dados Firebase

Baixa

Alto

Backups automaticos do Firestore

Latencia alta na transcricao de
audio

Baixa

Baixo

Groq Whisper processa em <3s

2.2 Viabilidade Economica

Custos de operacao estimados (mensal):

Item	Custo Estimado
VPS Hetzner (2 vCPU / 4 GB)	R\$ 80-120/mes
Firebase Firestore (Spark)	Gratuito ate cota
Groq API (LLaMA 70B)	Gratuito -- 14.400 req/dia
Gemini 2.0 Flash	Gratuito -- ~1M tokens/dia
Claude Haiku (fallback)	~R\$ 0-30/mes (uso marginal)
Evolution API	Gratuito (auto-hospedado)
Z-API (backup WhatsApp)	R\$ 97/mes (opcional)
Total minimo	~R\$ 80/mes

Modelo de receita: assinatura por fazenda (estimado R\$ 150-300/mes por cliente). Ponto de equilibrio: 1 cliente.

2.3 Viabilidade Operacional

O sistema foi projetado para operar com minima intervencao tecnica:

- * Deploy automatizado via GitHub Actions (CI/CD) a cada push na branch main.
- * Reinicializacao automatica do bot via systemctl em caso de falha.
- * Interface web sem instalacao, acessivel em qualquer navegador.
- * Bot WhatsApp nao exige instalacao de aplicativo pelo produtor.

2.4 Cronograma de Desenvolvimento

Fase	Descricao	Status
Fase 1	Modulos base: Rebanho, Ordenha, Financeiro,	Concluido
Fase 2	Bot WhatsApp com NLP, multi-tenant	Concluido
Fase 3	BI & Inteligencia, Bercario, Armazem,	Concluido
Fase 4	App Mobile (React PWA), API REST	Concluido
Fase 5	Protocolos Sanitarios, Painel Mobile	Concluido
Fase 6	CI/CD automatizado, infraestrutura Hetzner	Concluido
Fase 7	Expansao de clientes, monitoramento, SLA	Em andamento

3. Historias de Usuario

3.1 Bot WhatsApp

HU-01 -- Como produtor rural, quero enviar uma mensagem de texto simples pelo WhatsApp informando a producao do dia para que ela seja registrada sem precisar abrir nenhum aplicativo.

HU-02 -- Como produtor rural, quero enviar uma foto do quadro de producao (com varios animais e litros) para que o sistema reconheca e registre todos automaticamente.

HU-03 -- Como produtor rural, quero enviar um audio descrevendo uma compra de racao para que o sistema transcreva, entenda e registre no estoque.

HU-04 -- Como produtor rural, quero perguntar "quanto produzi essa semana?" pelo WhatsApp e receber a resposta com dados reais da fazenda.

HU-05 -- Como produtor rural, quero que o bot me avise quando eu perguntar "o que tenho pra fazer hoje?" sobre pendencias como ordenhas nao registradas, vacinas vencidas e partos previstos.

3.2 Rebanho e Reproducao

HU-06 -- Como gestor, quero cadastrar um novo animal (nascimento ou compra) com nome, raca, data de nascimento e mae, para manter o registro genealogico do rebanho.

HU-07 -- Como gestor, quero registrar uma inseminacao artificial com a data, tipo (IA ou touro) e tourador, para acompanhar o calendario reprodutivo.

HU-08 -- Como gestor, quero visualizar quais vacas estao em atraso de inseminacao (mais de 45 dias pos-parto sem inseminar) para tomar providencias.

HU-09 -- Como gestor, quero saber quais vacas precisam ser secadas nos proximos dias com base na data prevista de parto e no periodo de secagem configurado.

3.3 Controle de Ordenha

HU-10 -- Como operador, quero registrar a producao de uma vaca especifica (litros e racao consumida) para ter controle individual de produtividade.

HU-11 -- Como gestor, quero ver um heatmap de producao por dia e por animal para identificar padroes e quedas de produtividade.

HU-12 -- Como gestor, quero visualizar a curva de lactacao de cada vaca comparada ao modelo teorico (Wood) para identificar animais abaixo do esperado.

3.4 Sanidade e Protocolos

HU-13 -- Como gestor, quero registrar a aplicacao de um medicamento com nome, dose, animal (ou rebanho todo) e custo, para controlar gastos sanitarios.

HU-14 -- Como gestor, quero cadastrar um protocolo de vaccinacao recorrente (ex: aftosa a cada 6 meses) para que o sistema me lembre automaticamente da proxima aplicacao.

HU-15 -- Como gestor, quero executar um protocolo sanitario com um clique, registrando automaticamente o evento no historico e atualizando a proxima data.

3.5 Financeiro

HU-16 -- Como gestor, quero registrar a venda mensal de leite (litros e valor recebido) para controlar a receita da fazenda.

HU-17 -- Como gestor, quero ver o saldo financeiro do mes atual, com total de receitas e despesas, no painel financeiro.

HU-18 -- Como gestor, quero saber o custo por litro de cada vaca no mes para identificar animais nao rentaveis.

3.6 Armazem

HU-19 -- Como operador, quero registrar a entrada de racao no estoque (produto, quantidade, valor pago) para controlar

o inventario e o custo medio.

HU-20 -- Como operador, quero ver o estoque atual com quantidade disponivel e custo medio por item para planejar compras.

3.7 BI e Inteligencia

HU-21 -- Como gestor, quero ver um dashboard com os principais KPIs da fazenda (producao, receita, custo/L, saldo) para ter uma visao geral rapida.

HU-22 -- Como gestor, quero um ranking das vacas mais e menos lucrativas do mes para embasar decisoes de descarte ou investimento.

HU-23 -- Como gestor, quero uma previsao de producao para os proximos 7 dias baseada no historico recente.

3.8 Bercario

HU-24 -- Como operador, quero registrar o nascimento de um bezerro com data, mae, sexo e peso, para acompanhar seu desenvolvimento.

HU-25 -- Como operador, quero ver a lista de bezerros que ainda nao receberam colostro (ate 2 dias de vida) para nao perder o prazo critico.

3.9 Configuracoes

HU-26 -- Como administrador, quero associar um numero de WhatsApp a uma fazenda para habilitar o bot para aquele produtor.

HU-27 -- Como gestor, quero configurar o preco do leite, custo da racao e nome da fazenda para que os calculos de rentabilidade usem valores corretos.

4. Requisitos Funcionais

RF001 -- Cadastro de Animais

O sistema deve permitir o cadastro de animais com os campos: nome, raca, sexo, data de nascimento, data de entrada, mae (referencia a outro animal), status (Lactacao, Seca, Novilha, Bezerro, Vendido, Descartado) e observacoes.

RF002 -- Controle de Status Produtivo

O sistema deve controlar o status produtivo de cada animal, atualizando-o automaticamente quando eventos reprodutivos sao registrados (parto -> Lactacao, secagem -> Seca).

RF003 -- Registro de Ordenha Individual

O sistema deve registrar a producao de leite por animal por dia, incluindo: data, animal, litros, racao consumida (kg), turno (manha/tarde/noite) e observacoes.

RF004 -- Registro de Ordenha em Lote (PRODUCAO_MULTIPLA)

O sistema deve aceitar registros de multiplos animais em uma unica operacao, seja por texto formatado, audio, foto de quadro ou tabela mensal.

RF005 -- Interpretacao de Imagens com IA

O sistema deve usar visao computacional para interpretar fotos de quadros de producao (CASO A: multiplos animais/dia; CASO B: um animal/multiplas datas) e notas fiscais de compra.

RF006 -- Transcricao de Áudio

O sistema deve transcrever mensagens de voz recebidas pelo WhatsApp usando Groq Whisper, com fallback para OpenAI Whisper.

RF007 -- Bot WhatsApp com NLP

O sistema deve processar mensagens de texto em linguagem natural pelo WhatsApp, identificando a intencao do produtor e coletando os dados necessarios via dialogo conversacional.

RF008 -- Persistencia de Estado Conversacional

O bot deve manter o estado da conversa entre mensagens, incluindo tipo do registro em andamento, dados coletados parcialmente, e contexto do ultimo registro salvo (para evitar duplicatas).

RF009 -- Registro Financeiro

O sistema deve registrar lancamentos financeiros com: data, descricao, categoria (Venda de Leite, Venda de Animais, Racao/Nutricao, Medicamento, Infraestrutura, Mao de Obra, Energia, Compra de Animais, Outros), valor e tipo (receita/despesa).

RF010 -- Controle de Estoque

O sistema deve controlar o estoque de insumos (racao, medicamentos, etc.) com: produto, quantidade, unidade, custo unitario, custo medio ponderado historico.

RF011 -- Baixa Automatica de Estoque

O sistema deve dar baixa automatica no estoque quando um gasto sanitario e registrado com quantidade utilizada.

RF012 -- Registro de Eventos Reprodutivos

O sistema deve registrar: inseminacao artificial (data, tourador, tipo), cobertura natural, diagnostico de prenhez (positivo/negativo), parto (data, bezerro), secagem e aborto.

RF013 -- Calculo de Datas Reprodutivas

O sistema deve calcular automaticamente: data prevista de parto (inseminacao + 283 dias), data de secagem prevista (parto previsto ? 60 dias), data de diagnostico sugerida (inseminacao + 30 dias).

RF014 -- Agenda Automatica

O sistema deve gerar uma agenda diaria e semanal com: ordenhas pendentes, diagnosticos de prenhez vencidos, inseminacoes em atraso (PVE > 45 dias), secagens urgentes, partos previstos, colostro pendente, desmames.

RF015 -- Alertas Automaticos

O sistema deve gerar alertas classificados em "critico" e "atencao" visiveis no painel web e mobile, baseados na agenda automatica.

RF016 -- Registro de Sanidade

O sistema deve registrar aplicacoes de medicamentos/vacinas com: data, tipo (Vacina, Vermifugo, Antibiotico, Hormonio, Casqueamento, Outros), produto, modo (individual/rebanho todo), animal, custo e observacoes.

RF017 -- Protocolos Sanitarios Recorrentes

O sistema deve permitir o cadastro de protocolos periodicos com frequencia em dias, alvo (animal, lote ou rebanho todo) e custo estimado. A execucao atualiza automaticamente a proxima data e registra o evento no historico.

RF018 -- Dashboard de BI

O sistema deve exibir KPIs macro (producao total, receita, despesa, saldo, custo/L, lucro/L), graficos de producao diaria (barras), composicao de custos (pizza) e tendencia.

RF019 -- Raio-X Individual por Animal

O sistema deve calcular e exibir, por animal e por periodo: producao total, receita estimada, custo de racao, custo veterinario direto, custo fixo rateado, custo/L e lucro/L.

RF020 -- Ranking de Animais

O sistema deve ordenar os animais por diferentes metricas: producao total, custo/L, lucro/L, margem.

RF021 -- Curva de Lactacao

O sistema deve gerar a curva de lactacao de cada animal e compara-la ao modelo teorico de Wood (305 dias).

RF022 -- Previsao de Producao

O sistema deve gerar previsao de producao para os proximos 7 dias usando regressao polinomial sobre o historico.

RF023 -- Heatmap de Producao

O sistema deve exibir um heatmap com producao diaria por animal no periodo selecionado.

RF024 -- Calendario de Eventos

O sistema deve exibir um calendario mensal com eventos de reproducao, sanidade e partos, com filtros por tipo de evento.

RF025 -- Bercario

O sistema deve controlar bezerros (nascimento, mae, sexo, peso), registrar eventos (coloostro, vacinas de bezerro, pesagens) e alertar sobre desmame (90 dias de vida) e coloostro pendente (? 2 dias de vida).

RF026 -- Exportacao de Dados

O sistema deve permitir exportacao de relatorios em formato Excel (.xlsx) e PDF.

RF027 -- App Mobile (PWA)

O sistema deve fornecer um Progressive Web App React com login por PIN, consulta de KPIs, registro de producao e finanzas otimizados para toque.

RF028 -- API REST para Mobile

O sistema deve expor endpoints REST autenticados via JWT para o app mobile: login, dashboard, producao, financeiro, estoque, animais, agenda, alertas.

RF029 -- Multi-Tenant

O sistema deve suportar multiplas fazendas com dados completamente isolados no Firebase, identificadas por fazenda_id.

RF030 -- Autenticacao Web

O sistema deve exigir autenticacao para acesso as paginas web (Firebase Auth ou PIN configurado).

RF031 -- Configuracao por Fazenda

O sistema deve permitir configurar por fazenda: nome, preco do leite, custo da racao, periodo de secagem, periodo PVE.

RF032 -- Painel Mobile Streamlit

O sistema deve oferecer uma versao do painel principal otimizada para tela pequena (layout "centered"), com KPIs do dia, semana, mes, alertas e top vacas.

5. Requisitos Nao Funcionais

RNF001 -- Desempenho

- * O bot deve responder mensagens de texto simples em menos de 5 segundos.
- * Transcricao de audio deve ser concluida em menos de 10 segundos.
- * O painel web deve carregar dados iniciais em menos de 3 segundos.

RNF002 -- Disponibilidade

- * O sistema deve ter disponibilidade minima de 95% por mes.
- * O bot deve retomar operacao automaticamente em caso de reinicializacao (systemctl restart).

RNF003 -- Escalabilidade

- * A arquitetura multi-tenant com Firebase Firestore deve suportar expansao para centenas de fazendas sem alteracao de codigo.
- * A cascata de IA (Groq -> Gemini -> Claude) garante operacao mesmo com indisponibilidade de fornecedores individuais.

RNF004 -- Seguranca

- * Chaves de API (ANTHROPIC_API_KEY, GROQ_API_KEY, etc.) devem ser armazenadas em variaveis de ambiente, nunca em codigo ou repositorio.
- * O arquivo firebase_key.json deve constar no .gitignore.
- * Tokens JWT para o app mobile devem expirar em 30 dias com validacao de assinatura HMAC-SHA256.
- * PINs de usuario mobile armazenados como hash SHA-256.

RNF005 -- Usabilidade

- * O bot deve entender mensagens com erros ortograficos, girias e regionalismos do produtor rural.
- * O produtor nao deve precisar seguir um formato rigido de mensagem.
- * O sistema deve pedir confirmacao antes de salvar qualquer registro.

RNF006 -- Manutenibilidade

- * O codigo Python deve seguir organizacao modular: utils.py (funcoes compartilhadas), whatsapp_bot.py (bot + API), pages/ (uma pagina por modulo).
- * Deploy automatizado via GitHub Actions a cada push em main.

RNF007 -- Compatibilidade

- * O app web deve funcionar nos navegadores Chrome, Firefox, Edge e Safari (ultimas 2 versoes).
- * O PWA mobile deve funcionar em Android 9+ e iOS 14+.

RNF008 -- Portabilidade

- * O sistema deve poder ser migrado entre provedores de VPS sem alteracao de codigo.
- * Toda configuracao sensivel gerenciada via variaveis de ambiente.

RNF009 -- Confiabilidade dos Dados

- * Todos os registros persistidos no Firebase devem incluir timestamp de criacao.
- * O bot nao deve salvar dados sem confirmacao explicita do produtor ("sim" / "confirmar").
- * O bot deve preservar contexto do ultimo registro salvo para evitar duplicatas em seguimento de conversa.

RNF010 -- Privacidade

- * Dados de cada fazenda devem ser isolados por fazenda_id, sem acesso cruzado.
- * Numeros de telefone sao a chave de identificacao do produtor e nao devem ser expostos em logs publicos.

6. Casos de Uso

6.1 Atores

Ator	Descricao
Produtor	Usuario final que interage pelo WhatsApp ou App Mobile
Gestor	Usuario com acesso ao painel web completo
Administrador	Configura o sistema e gerencia cadastros
Bot IA	Sistema automatizado que processa mensagens
Firebase	Sistema externo de persistencia
LLM	Sistema externo de inteligencia artificial (Groq/Gemini/Claude)

6.2 Diagrama de Casos de Uso -- Bot WhatsApp



```

|               Sistema MilkShow -- Bot               |
|
| +-----+
| | UC01: Registrar Producao |<-----+
| +-----+
| +-----+
| | UC02: Registrar Compra  |<-----+
| +-----+
| +-----+ | <<uses>> |
| | UC03: Registrar Gasto   |<-----+
| +-----+ [Produtor]-->|
| +-----+
| | UC04: Registrar Reproducao |<-----+
| +-----+
| +-----+
| | UC05: Consultar Fazenda  |<-----+
| +-----+
| +-----+
| | UC06: Enviar Foto/Áudio  |<-----+
| +-----+
|
| +-----+
| | UC07: Processar com IA   | <-- [LLM] (Groq/Gemini/Claude) |
| +-----+
| +-----+
| | UC08: Persistir Dados   | <-- [Firebase]
| +-----+
+-----+

```

6.3 Diagrama de Casos de Uso -- Painel Web

```

+-----+
|               Sistema MilkShow -- Web               |
|
| +-----+
| | UC10: Acessar Dashboard BI |<-----+
| +-----+
| +-----+
| | UC11: Gerenciar Rebanho   |<-----+
| +-----+
| +-----+
| | UC12: Gerenciar Financeiro |<-----+ [Gestor]
| +-----+
| +-----+
| | UC13: Controlar Sanidade  |<-----+
| +-----+
| +-----+
| | UC14: Gerenciar Estoque   |<-----+
| +-----+
| +-----+
| | UC15: Configurar Sistema  |<-----+
| +-----+ [Administrador]-->UC15|
+-----+

```

7. Descricao de Casos de Uso

UC01 -- Registrar Producao pelo WhatsApp

Atores: Produtor, Bot IA, Firebase

Pre-condicao: Numero de telefone cadastrado e associado a uma fazenda.

Pos-condicao: Registro de producao salvo na colecao producao do Firebase.

Fluxo Principal:

1. Produtor envia mensagem de texto (ex: "Joana deu 22 litros hoje manha").
2. Bot normaliza o numero de telefone e identifica a fazenda (`_find_fazenda`).
3. Bot carrega contexto: lista de animais, estoque, dados da fazenda.
4. Bot envia ao LLM (Grok -> Gemini -> Claude) o historico + system prompt.
5. LLM identifica intencao `PRODUCAO_LEITE` e coleta campos necessarios.
6. LLM retorna JSON `{"tipo":"PRODUCAO_LEITE", "estado":"CONFIRMANDO", "dados":{"..."}, "texto":"Confirma? Joana -- 22L, manha."}`.
7. Bot envia confirmacao ao produtor.
8. Produtor responde "sim".
9. Bot salva registro no Firebase e responde com resumo.
10. Bot reseta conversa, preservando contexto do ultimo registro.

Fluxo Alternativo A -- Multiplas Vacas:

- 3a. Produtor envia "Joana 22, Moreninha 18, Pintada 15" ou foto de quadro.
- 3b. Bot identifica `PRODUCAO_MULTIPLA` com lista de animais (CASO A: multiplos animais/mesmo dia).
- 3c. Bot salva um registro por animal na mesma transacao.

Fluxo Alternativo B -- Tabela Mensal:

- 3a. Produtor envia foto de tabela com datas e litros de uma vaca.
- 3b. Vision IA classifica como [TABELA MENSAL] (CASO B: um animal/multiplas datas).
- 3c. Bot coleta o nome do animal (nao o cabecalho da tabela).
- 3d. Bot salva um registro por linha da tabela.

Fluxo de Excecao:

- * Se o animal citado nao existe no rebanho -> bot avisa e pergunta se quer cadastra-lo.
- * Se litros > 1.000 por animal -> bot questiona o valor.

UC02 -- Registrar Compra de Produto

Atores: Produtor, Bot IA, Firebase

Pre-condicao: Numero cadastrado.

Pos-condicao: Item adicionado/atualizado na colecao estoque com custo medio ponderado. Lancamento de despesa criado em financeiro.

Fluxo Principal:

1. Produtor envia "comprei 5 sacos de racao por R\$300".
2. LLM identifica `COMPRA_PRODUTO`: `produto="racao", qtd=5, valor_unit=60`.
3. Bot confirma com produtor.
4. `_salvar` atualiza o estoque com custo medio ponderado: $(qtd_atual \times custo_atual + qtd_nova \times custo_novo) / (qtd_atual + qtd_nova)$.
5. Lancamento financeiro registrado como despesa categoria "Racao / Nutricao".

Fluxo Alternativo -- Nota Fiscal:

- 2a. Produtor envia foto de nota fiscal.
 - 2b. Claude Vision extrai: fornecedor, data, lista de itens com quantidade e valor.
 - 2c. Bot processa como multiplos COMPRA_PRODUTO (array itens).
-

UC03 -- Consultar Dados da Fazenda

Atores: Produtor, Bot IA, Firebase

Pre-condicao: Numero cadastrado.

Pos-condicao: Resposta com dados reais enviada ao produtor.

Fluxo Principal:

1. Produtor envia "quanto produzi essa semana?".
 2. Bot detecta palavra-chave de consulta, carrega contexto completo (_ctx_dados_fazenda).
 3. LLM recebe contexto com: rebanho, producao dos ultimos 7 dias, financeiro do mes, estoque, reproducao recente, agenda e rentabilidade por animal.
 4. LLM gera resposta em linguagem natural com dados especificos.
 5. Estado retorna CONSULTA -- historico mantido para follow-up.
-

UC10 -- Acessar Dashboard de BI

Atores: Gestor

Pre-condicao: Usuario autenticado no painel web.

Pos-condicao: Dashboard exibido com dados do periodo selecionado.

Fluxo Principal:

1. Gestor acessa a pagina "BI e Inteligencia".
 2. Sistema carrega dados do Firebase (animais, producao, financeiro).
 3. Sistema calcula KPIs usando calcular_bi(df_prod, df_fin, ini, fim).
 4. Exibe: producao total, receita estimada, despesa, saldo, custo/L, lucro/L.
 5. Exibe graficos: barras de producao diaria, pizza de composicao de custos.
 6. Disponibiliza aba Rankings com scatter custo vs volume.
 7. Gestor pode exportar relatorio em Excel ou PDF.
-

UC14 -- Executar Protocolo Sanitario

Atores: Gestor

Pre-condicao: Protocolo cadastrado na lista de protocolos ativos.

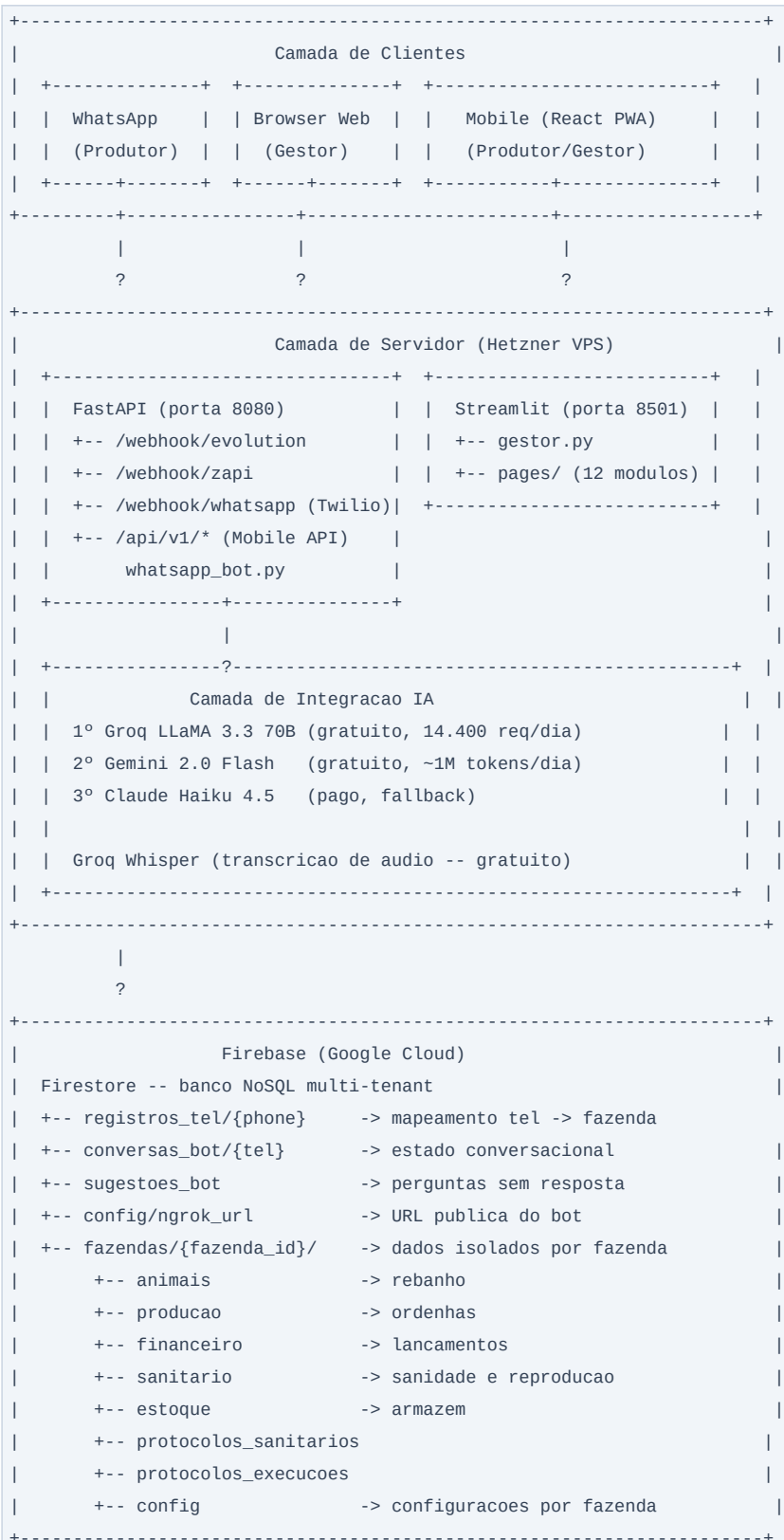
Pos-condicao: Execucao registrada em protocolos_execucoes, evento lancado em sanitario, proxima data atualizada.

Fluxo Principal:

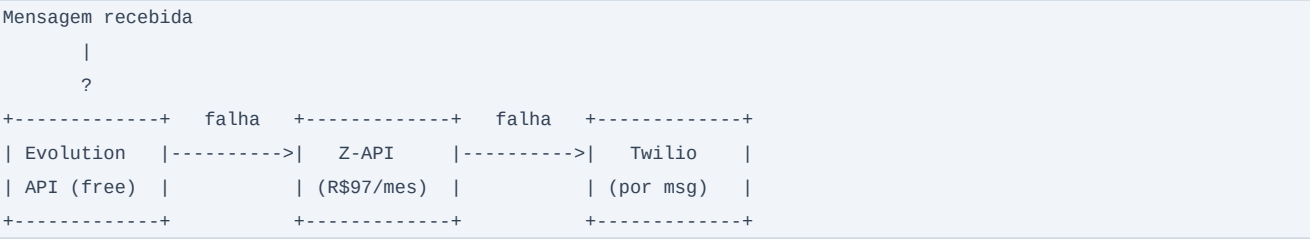
1. Gestor acessa pagina "Protocolos Sanitarios" -> aba "Protocolos Ativos".
 2. Sistema lista protocolos ordenados por proximidade da data de aplicacao, com badge de cor (vencido/hoje/em breve/ok).
 3. Gestor clica "Executar" no protocolo desejado.
 4. Sistema registra execucao em protocolos_execucoes.
 5. Sistema lanca evento em sanitario (tipo, produto, modo, custo).
 6. Sistema atualiza proxima_data = hoje + frequencia_dias.
 7. Sistema exibe confirmacao e recarrega a pagina.
-

8. Arquitetura de Alto Nivel

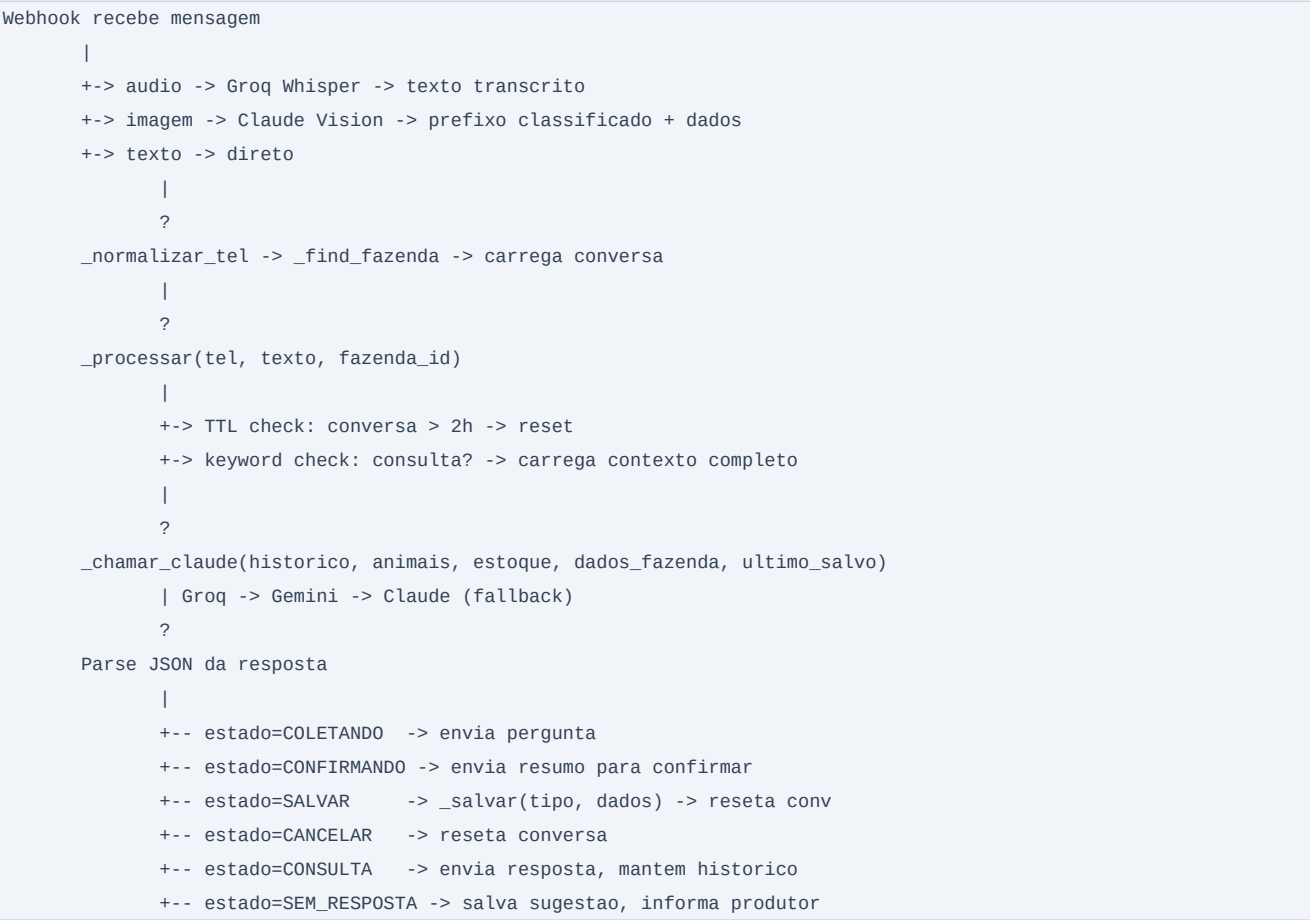
8.1 Visao Geral



8.2 Cascata de WhatsApp



8.3 Fluxo de Processamento do Bot



8.4 Padroes Arquiteturais

Padrao	Aplicacao
Multi-tenant	Dados isolados por fazenda_id no Firestore
Cascata	IA e WhatsApp com multiplos provedores em fallback
State Machine	Estados de conversa do bot (idle/coletando/confirmando/salvar/...)
Repository	_coll(fazenda_id, nome) isola acesso ao Firestore
Cache	@st.cache_resource para conexao Firebase; TTL de 60s para dados
Strategy	Alternancia entre provedores de IA conforme disponibilidade

8.5 Infraestrutura de Deploy

GitHub (main branch)

```
|
| push
?
```

GitHub Actions (ubuntu-latest)

```
|
| appleboy/ssh-action
?
```

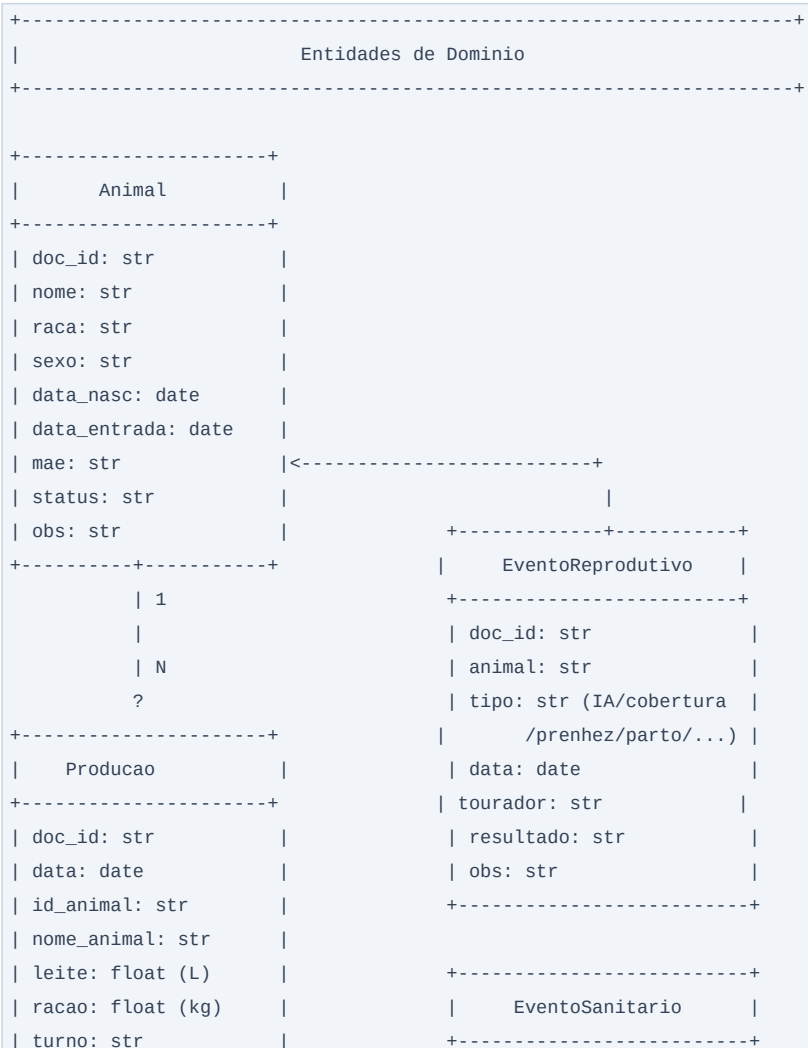
Hetzner VPS (178.104.252.193)

```
|
+-- git checkout origin/main -- [arquivos relevantes]
+-- systemctl restart milkshow-bot
+-- verifica: systemctl is-active milkshow-bot
```

Processos ativos no VPS:

```
+-- milkshow-bot.service -> uvicorn whatsapp_bot:app --port 8080
+-- streamlit              -> streamlit run gestor.py --port 8501
```

9. Diagrama de Classes



```

| obs: str | | doc_id: str |
+-----+ | data: date |
| | | tipo: str |
+-----+ | prod: str |
| Lancamento | | modo: str (Individual/
+-----+ | Rebanho Todo) |
| doc_id: str | | animal: str |
| data: date | | custo: float |
| descricao: str | | obs: str |
| cat: str | +-----+
| valor: float |
| tipo: str (rec/desp) | +-----+
+-----+ | ItemEstoque |
| | +-----+
+-----+ | doc_id: str |
| ProtocoloSanitario | | item: str |
+-----+ | qtd: float |
| doc_id: str | | un: str |
| nome: str | | custo_unit: float |
| tipo: str | | custo_medio: float |
| frequencia_dias: int | +-----+
| proxima_data: date |
| ultima_data: date | +-----+
| alvo: str | | Conversa Bot |
+-----+ |
| custo_estimado: float | +-----+
| obs: str | | tel: str (PK) |
| ativo: bool | | historico: list[dict] |
+-----+ | estado: str |
| | | dados: dict |
| | | tipo: str |
+-----+ | itens: list |
| RegistroTelefone | | ultimo_salvo: dict |
+-----+ | ts: datetime |
| phone: str (PK) | +-----+
| fazenda_id: str |
| nome: str |
| ativo: bool |
| pin_hash: str |
| permissoes: list |
+-----+

```

9.1 Modulos Python

```

+-----+
|                               |
|                               |
+-----+
| CONSTANTES |
| DIAS_PVE=45, DIAS_DIAGNOSTICO=30, DIAS_SECAGEM=60 |
| GESTACAO=283, DIAS_DESMAME=90 |
| PRECO_PADRAO_LEITE=2.50, FATOR_CONVERSAO=3.0 |
| CATEGORIAS_FINANCEIRAS, TIPOS_SANITARIOS |
| |
| FUNÇÕES |
| init_firebase() -> singleton Firebase Admin |
| carregar_dados() -> carrega colecoes no session_state |

```

```

| get_config(chave, default) -> le configuracao da fazenda |
| get_custo_racao() -> custo medio ponderado do estoque |
| calcular_bi(df_prod, df_fin, ini, fim) -> KPIs macro |
| processar_alertas() -> lista alertas criticos/atencao |
| apply_theme() -> CSS dark theme |
| page_banner(icon, title, subtitle) -> cabecalho padrao |
| requer_autenticacao() -> guard de autenticacao |
+-----+
+-----+
|                                     whatsapp_bot.py |
+-----+
| FastAPI app |
| |
| FUNÇÕES AUXILIARES |
| _normalizar_tel(raw) -> str |
| _variantes_tel(tel) -> list[str] |
| _find_fazenda(tel) -> str (fazenda_id) |
| _coll(fazenda_id, nome) -> CollectionReference |
| _ctx_animais(fazenda_id) -> str |
| _ctx_estoque(fazenda_id) -> str |
| _ctx_dados_fazenda(fazenda_id) -> str (contexto rico) |
| _buscar_no_estoque(fazenda_id, nome) -> dict|None |
| _parse_float(v) -> float |
| _clear_conv(tel, ultimo_salvo=None) |
| _get_conv(tel) -> dict |
| |
| ENVIO WHATSAPP (cascata) |
| _enviar_evolution(para, msg) -> bool |
| _enviar_zapi(para, msg) -> bool |
| _enviar_twilio(para, msg) -> bool |
| _enviar_whatsapp(para, msg) -> bool |
| |
| IA (cascata) |
| _ia_groq(system, historico) -> str|None |
| _ia_gemini(system, historico) -> str|None |
| _ia_claude(system, historico) -> str|None |
| _chamar_claude(hist, animais, estoque, dados_fazenda, |
|                 memoria, ultimo_salvo) -> str |
| |
| MULTIMIDIA |
| _transcrever(audio_url, content_type) -> str |
| _ler_imagem(image_url, animais_str) -> str |
| |
| PROCESSAMENTO |
| _processar(tel, texto, fazenda_id) -> str |
| _salvar(tipo, dados, fazenda_id) -> str |
| |
| ENDPOINTS |
| POST /webhook/whatsapp (Twilio) |
| POST /webhook/evolution |
| POST /webhook/zapi |
| GET /status |
+-----+
+-----+

```

```

|                                     mobile_api.py                                     |
+-----+
| APIRouter prefix="/api/v1"          |
|                                     |
| AUTH                                |
|   POST /auth/login      -> JWT com fazenda_id, tel, permissoes |
|   POST /auth/google     -> Google ID Token -> JWT              |
|                                     |
| DADOS (requer JWT)                |
|   GET  /dashboard       -> KPIs, ultimas producoes, alertas   |
|   GET  /animais         -> lista animais ativos               |
|   POST /producao        -> registrar ordenha                  |
|   GET  /producao        -> historico de producao               |
|   GET  /financeiro      -> lancamentos do mes                  |
|   POST /financeiro      -> registrar lancamento               |
|   GET  /estoque         -> lista estoque                       |
|   POST /estoque         -> registrar entrada                  |
|   GET  /agenda          -> tarefas pendentes                  |
|   GET  /alertas         -> alertas ativos                      |
+-----+

```

10. Diagrama de Sequencias

10.1 Fluxo Completo de Registro por WhatsApp (Producao)

Produtor	Evolution API	FastAPI(bot)	Groq LLM	Firebase
--"Joana 22L"-->				
	--POST /webhook/evolution-->			
		--_find_fazenda()----->		
		<-fazenda_id-----		
		--_get_conv()----->		
		<-historico + estado-----		
		--_ctx_animais()----->		
		<-lista animais-----		
		--_ia_groq(system, hist)----->		
		<-JSON {tipo:PRODUCAO_LEITE,		
		estado:CONFIRMANDO,		
		dados:{animal:Joana,22L}}--		
		--salva estado CONFIRMANDO----->		
	<--"Confirma? Joana -- 22L"-----			
<--"Confirma? Joana -- 22L"----				
--"sim"----->				
	--POST /webhook/evolution-->			
		--_ia_groq(...)----->		
		<-{estado:SALVAR}-----		

```
|
|
|      |--salvar(PRODUCAO_LEITE, dados)-->
|
|      |<-doc_id criado-----|
|
|      |
|      |--clear_conv(tel, ultimo_salvo)-->
|
|      |<--"Joana: 22L registrado!"-----|
|
|<--"Joana: 22L registrado!"---|
|
|
|
```

10.2 Fluxo de Foto de Nota Fiscal

Produtor	Evolution API	FastAPI(bot)	Claude Vision	Firebase
--[foto NF]---->				
	--POST /webhook/evolution-->			
		--_ler_imagem(url, animais)----->		
		<- "[NOTA FISCAL] Fornecedor: X		
		Item1: racao 50kg R\$300		
		Item2: vacina 10ml R\$50"		
		--_processar("prefixo...", faz)		
		--_ia_groq(system, hist)----->(Groq)		
		<- {tipo:COMPRA_PRODUTO,		
		estado:CONFIRMANDO,		
		itens:[{racao,50kg,R\$300},		
		{vacina,10ml,R\$50}]]		
	<-- "2 itens: racao 50kg + vacina. Confirma?"-----			
<-- "Confirma?"-----				
--"sim"----->				
		--_salvar (loop itens)----->		
<-- "2 itens salvos no estoque."				

10.3 Fluxo de Autenticacao Mobile

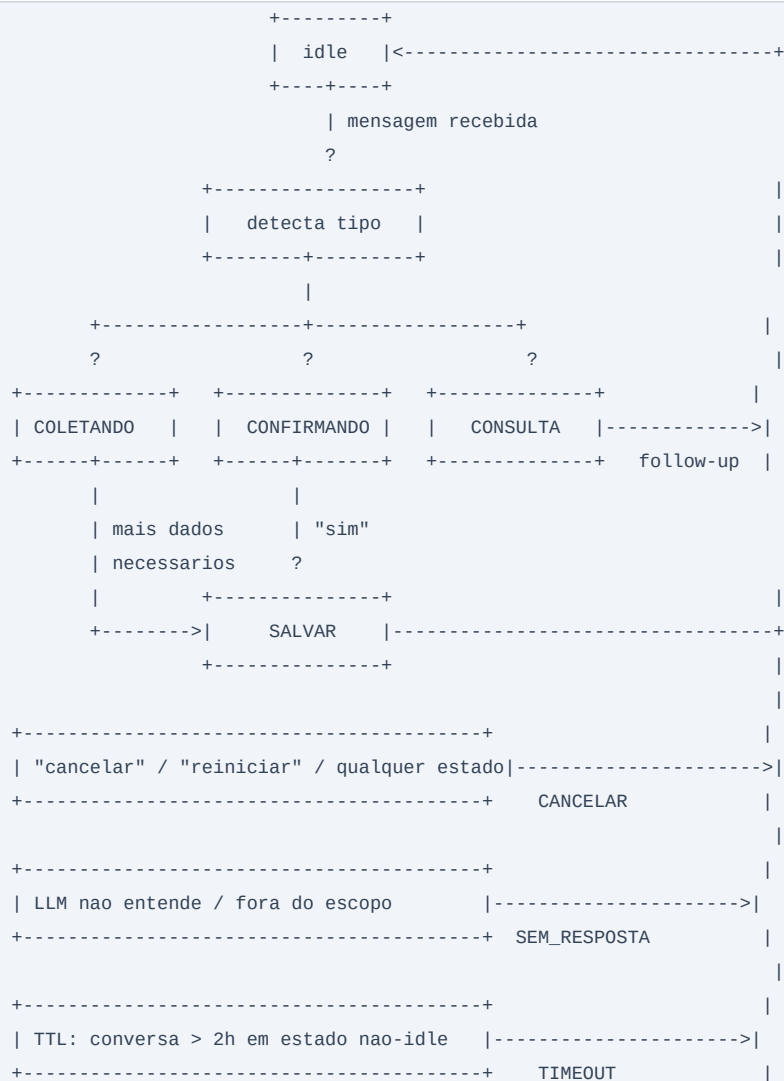
```

App Mobile           FastAPI /api/v1           Firebase
|                   |                   |
|--POST /auth/login |                   |
|   {tel, pin}----->                   |
|                   |--busca registros_tel/{tel}-->|
|                   |<--{{fazenda_id, pin_hash}}-----|
|                   |                   |
|                   | verifica SHA256(pin)==pin_hash
|                   |                   |
|                   |--gera JWT {tel, fazenda_id, exp=+30d}
|<--{{token, nome, fazenda_id}}-----|
|                   |                   |
|--GET /dashboard   |                   |
|   Authorization: Bearer <token>----->
|                   | valida JWT (HMAC-SHA256)
|                   |--carrega dados fazenda----->|
|<--{{kpis, alertas, animais}}-----|

```

11. Diagrama de Estados

11.1 Estados da Conversa do Bot



11.2 Estados de Status do Animal





11.3 Ciclo de Vida de um Protocolo Sanitario



12. Plano de Testes

12.1 Testes Funcionais do Bot

ID	Caso de Teste	Entrada	Resultado Esperado
T001	Registro producao texto simples	"Joana 22 litros"	Estado CONFIRMANDO, dados={animal:Joana, leite:22}

T002

Producao com erro ortografico

"joaninha deu 20 litro"

Identifica animal mais proximo,
coleta confirmacao

T003

Producao multipla texto

"Joana 22, Moreninha 18, Pintada
15"

PRODUCAO_MULTIPLA com 3 itens

T004

Foto quadro producao (CASO A)

[imagem: 3 vacas e litros]

Classifica [PRODUCAO DO DIA],
extraí animais e litros

T005

Foto tabela mensal (CASO B)

[imagem: 1 vaca / 30 datas]

Classifica	[TABELA	MENSAL],
pede nome do animal		

T006

Nome	na	tabela	?	animal
cadastrado				

Cabecalho "Fobinha" (produtor)

Nao assume como animal, usa
"Rebanho"

T007

Compra de racao por audio

[audio: "comprei 5 sacos de racao
por 300"]

Transcreve,	extraí
produto/qtd/valor	

T008

Foto nota fiscal

[imagem: NF com 3 itens]

Extrai fornecedor, 3 itens com qtd
e valor

T009

Consulta producao semanal

"quanto produzi essa semana?"

Resposta com litros dos ultimos 7 dias

T010

Agenda do dia

"o que tenho pra fazer hoje?"

Lista	ordenhas	pendentes,
secagens,	partos	

T011

Cancelamento

"cancelar" em qualquer estado

Estado reset para idle

T012

Timeout 2h

conversa sem resposta > 2h

Reset automatico ao receber
proxima mensagem

T013

Duplicata pos-salvar

"usei todos em abril" apos compra
salva

NAO registra de novo, usa
contexto ultimo_salvo

T014

Compra multi-item

"comprei milho 60kg R\$180 e soja
40kg R\$120"

2	lancamentos	separados	no
	estoque		

T015

Animal nao cadastrado

"Margarida inexistente)	18L"	(animal
----------------------------	------	---------

Avisa que animal nao existe,
pergunta se cadastra

12.2 Testes de Persistencia

ID	Caso de Teste	Resultado Esperado
T020	Custo medio ponderado no estoque	Entrada: 10kg@2,00 + 10kg@3,00 ->
T021	Calculo de data prevista de parto	IA em 01/01/2026 -> parto previsto em 10/10/2026 (283 dias)
T022	Data de secagem prevista	Parto 10/10 -> secar em 11/08 (60 dias antes)
T023	Isolamento multi-tenant	Dados de fazenda_A nao visiveis para
T024	Protocolo execucao atualiza data	Protocolo freq=180d executado hoje -> proxima_data=hoje+180

12.3 Testes de Interface

ID	Caso de Teste	Resultado Esperado
T030	Dashboard BI -- sem dados	Mensagem informativa sem erros Python
T031	Exportacao Excel	Arquivo .xlsx baixado com dados do periodo
T032	Exportacao PDF	Arquivo .pdf baixado com relatorio formatado
T033	Calendario -- filtro por tipo	Somente eventos do tipo selecionado exibidos
T034	Heatmap -- periodo sem dados	Heatmap vazio sem erro
T035	Protocolo vencido	Badge vermelho exibido, botao "Executar" disponivel

12.4 Testes de Seguranca

ID	Caso de Teste	Resultado Esperado
T040	JWT expirado	GET /api/v1/dashboard -> 401 Unauthorized
T041	JWT invalido (tampered)	GET /api/v1/dashboard -> 401 Unauthorized
T042	PIN incorreto no mobile	POST /auth/login -> 401 "PIN incorreto"
T043	Tel nao cadastrado	POST /auth/login -> 404 "Numero nao
T044	firebase_key.json no git	Verificar .gitignore inclui firebase_key.json

12.5 Testes de Integracao

ID	Caso de Teste	Resultado Esperado
T050	GitHub Actions deploy	Push em main -> Actions verde -> bot reiniciado no VPS
T051	Evolution webhook recebe e responde	Envio real pelo WhatsApp -> resposta < 5s
T052	Fallback Groq -> Gemini	Simular falha do Groq -> Gemini responde
T053	Transcricao audio real	Envio de audio .ogg pelo WhatsApp -> transcricao correta

13. Telas do Sistema

13.1 Estrutura de Navegacao

```
gestor.py (Home / Login)
+-- 1_?_BI_e_Inteligencia.py
|   +-- Tab: Dashboard Geral
|   +-- Tab: Raio-X Individual
|   +-- Tab: Relatorio Comparativo
|   +-- Tab: Rankings
|   +-- Tab: Curva de Lactacao
|   +-- Tab: Previsao
|   +-- Tab: Heatmap
+-- 2_?_Veterinaria.py
|   +-- Tab: Reproducao (registro de eventos)
|   +-- Tab: Historico Reprodutivo
+-- 3_?_Sanidade.py
|   +-- Tab: Registros Sanitarios
|   +-- Tab: Historico
+-- 4_?_Bercario.py
|   +-- Tab: Bezerros Ativos
|   +-- Tab: Registros de Desenvolvimento
|   +-- Tab: Alertas do Bercario
+-- 5_?_Financeiro_360.py
|   +-- Tab: Lancamentos
|   +-- Tab: Resumo Mensal
|   +-- Tab: Venda de Leite
+-- 6_?_Armazem.py
|   +-- Tab: Estoque Atual
|   +-- Tab: Movimentacoes
+-- 7_?_Rebanho_Geral.py
|   +-- Tab: Lista de Animais
|   +-- Tab: Cadastrar Animal
|   +-- Tab: Score de Saude do Rebanho
+-- 8_?_Calendario.py
|   +-- Calendario mensal com filtros
+-- 9_?_Controle_de_Ordenha.py
|   +-- Tab: Registrar Ordenha
|   +-- Tab: Historico
|   +-- Tab: Analise por Animal
+-- 10_??_Configuracoes.py
|   +-- Tab: Fazenda (nome, preco leite, etc.)
|   +-- Tab: Bot IA (cadastro de tel, iniciar/parar)
|   +-- Tab: Sistema (backup, reset)
+-- 11_?_Painel_Mobile.py
|   +-- KPIs compactos + alertas (layout centered)
+-- 12_?_Protocolos_Sanitarios.py
|   +-- Tab: Protocolos Ativos
|   +-- Tab: Novo Protocolo
|   +-- Tab: Historico de Execucoes
```

13.2 Padrao Visual das Telas

Todas as telas seguem o padrao dark theme definido em `apply_theme()` no `utils.py`:

- * Fundo: #0f172a (azul muito escuro)
- * Cards: `rgba(255,255,255,0.04)` com borda sutil

- * Texto principal: #f1f5f9
- * Texto secundario: #94a3b8
- * Acento azul: #3b82f6
- * Sucesso/verde: #4ade80
- * Atencao/amarelo: #ffbf24
- * Critico/vermelho: #f87171
- * Fonte: Inter (Google Fonts)

13.3 Componentes Reutilizaveis

Componente	Funcao	Exemplo de Uso
page_banner(icon, title, sub)	Cabecalho com gradiente e subtítulo	Todas as paginas
st.metric(label, value, delta)	KPI card nativo Streamlit	Dashboards
st.dataframe(df, use_container_width=True)	Tabela de dados responsiva	Historicos
st.plotly_chart(fig, use_container_width=True)	Graficos interativos Plotly	BI
Badges HTML	Tags coloridas de status	Protocolos, alertas
st.expander	Secoes colapsaveis	Inativos, detalhes

13.4 Tela -- Dashboard BI (Raio-X Individual)

+-----+																							
MilkShow BI e Inteligencia		[? logo]																					
+-----+																							
[Dashboard Geral]		[Raio-X Individual]		[Rankings]		[Curva]		[...]															
+-----+																							
Animal: [Joana ?]		Periodo: [01/01/2026]		a [30/04/2026]																			
+-----+																							
+-----+		+-----+		+-----+		+-----+																	
820 L		R\$2,50/L		R\$2.050		R\$430																	
Producao		Preco/L		Receita		Custo																	
+-----+		+-----+		+-----+		+-----+																	
+-----+		+-----+																					
R\$0,52/L		R\$1,98/L																					
Custo/L		Lucro/L		? Margem: 79%																			
+-----+		+-----+																					
===== Leite vs Racao (area) =====																							
? Leite (eixo esq)		? Racao (eixo dir)																					
[grafico de areas sobrepostas com gradiente]																							
=====																							
+-----+																							

13.5 Tela -- Protocolos Sanitarios

+-----+									
MilkShow Protocolos Sanitarios		[? logo]							

```

+-----+
| [Protocolos Ativos] [Novo Protocolo] [Historico de Execucoes] |
+-----+
|
| Vacina Aftosa [Vacina] |
| ? Proxima: 2026-05-10 [HOJE ?] A cada 180d | Rebanho todo |
| [Executar] |
| ----- |
| Vermifugacao [Vermifugo] |
| ? Proxima: 2026-05-28 [em 22d ?] A cada 90d | Rebanho todo |
| [Executar] |
| ----- |
| Casqueamento Joana [Casqueamento] |
| ? Proxima: 2026-07-01 [em 56d ?] A cada 120d | Joana |
| [Executar] |
|
| ? Inativos (2) |
+-----+

```

13.6 Tela -- Painei Mobile

```

+-----+
| Bom dia! ? |
| Fazenda Sao Joao -- 06/05/2026 |
| ----- |
| Hoje |
| +-----+ +-----+ +-----+ |
| | 350L | | 8/10 | | R$875 | |
| |Litros| |Orden.| |Receita | |
| +-----+ +-----+ +-----+ |
| ----- |
| Semana |
| Total 7d: 2.450 L Media: 350L |
| ----- |
| Mes Atual |
| Receitas: R$10.500 |
| Despesas: R$7.200 |
| Saldo: R$3.300 ^ |
| ----- |
| Alertas |
| ? URGENTE |
| 2 vacas nao ordenhadas hoje |
| ----- |
| Top Vacas -- Hoje |
| Joana ##### 38L |
| Moreninha ##### 30L |
| Pintada ##### 22L |
| ----- |
| [ Atualizar dados ] |
+-----+

```

13.7 Tela -- Bot WhatsApp (exemplo de dialogo)

```

+-----+

```

```

| WhatsApp -- MilkShow Bot |
+-----+
|
| Produtor:
| +-----+
| | "Joana 22 litros, Moreninha 18, Pintada 15" |
| +-----+
|
| MilkShow Bot:                ??
| +-----+
| | Confirma? 3 vacas -- 55L total |
| | * Joana: 22L |
| | * Moreninha: 18L |
| | * Pintada: 15L |
| +-----+
|
| Produtor:
| +-----+
| | "sim" |
| +-----+
|
| MilkShow Bot:                ??
| +-----+
| | ? 3 registros salvos! (55L no total) |
| +-----+
|
+-----+

```

Apendice A -- Colecoes do Firebase

Colecao	Escopo	Campos Principais
registros_tel/{phone}	Global	fazenda_id, nome, ativo, pin_hash, permissoes
conversas_bot/{tel}	Global	historico, estado, dados, tipo, itens, ultimo_salvo, ts
sugestoes_bot	Global	fazenda_id, pergunta, ts
config/ngrok_url	Global	url
fazendas/{id}/animais	Por fazenda	nome, raca, sexo, data_nasc, data_entrada, mae, status, obs
fazendas/{id}/producao	Por fazenda	data, id_animal, nome_animal, leite, racao, turno, obs
fazendas/{id}/financeiro	Por fazenda	data, descricao, cat, valor, tipo, animal
fazendas/{id}/sanitario	Por fazenda	data, tipo, prod, modo, animal, custo, obs
fazendas/{id}/estoque	Por fazenda	item, qtd, un, custo_unit, custo_medio,
fazendas/{id}/config	Por fazenda	preco_leite, custo_racao, nome_fazenda, ...
fazendas/{id}/protocolos_sanitarios	Por fazenda	nome, tipo, frequencia_dias, proxima_data, ultima_data, alvo, custo_estimado, ativo
fazendas/{id}/protocolos_execucoes	Por fazenda	protocolo_id, protocolo_nome, data, obs

Apendice B -- Variaveis de Ambiente

Variavel	Obrigatoria	Descricao
ANTHROPIC_API_KEY	Sim	Chave Claude Haiku (fallback de IA)
GROQ_API_KEY	Sim	Chave Groq LLaMA + Whisper (IA principal)
GOOGLE_API_KEY	Sim	Chave Gemini 2.0 Flash (IA secundaria)
NGROK_AUTHTOKEN	Sim (dev)	Token para tunnel ngrok local
EVOLUTION_URL	Recomendado	URL do servidor Evolution API
EVOLUTION_KEY	Recomendado	API Key do Evolution API
EVOLUTION_INSTANCE	Recomendado	Nome da instancia Evolution
ZAPI_INSTANCE	Opcional	Instancia Z-API (backup)
ZAPI_TOKEN	Opcional	Token Z-API
ZAPI_CLIENT_TOKEN	Opcional	Client token Z-API
TWILIO_ACCOUNT_SID	Opcional	Account SID Twilio (ultimo recurso)
TWILIO_AUTH_TOKEN	Opcional	Auth Token Twilio
TWILIO_FROM	Opcional	Numero Twilio (ex: whatsapp:+14155238886)
BOT_ADMIN_TOKEN	Recomendado	Secret para assinatura JWT do app mobile

Apendice C -- Constantes de Negocio

Constante	Valor	Descricao
DIAS_PVE	45	Periodo Voluntario de Espera pos-parto
DIAS_DIAGNOSTICO	30	Dias para diagnostico de prenhez pos-IA
DIAS_SECAGEM	60	Periodo de secagem antes do parto
GESTACAO	283	Dias de gestacao bovina
DIAS_DESMAME	90	Dias de vida ate o desmame do bezerro
PRECO_PADRAO_LEITE	2.50	Preco padrao R\$/L (quando nao configurado)
DENSIDADE_LEITE	1.032	Densidade do leite (kg/L)
BASE_PASTO_LITROS	4.0	Producao base de pasto (L/dia)
FATOR_CONVERSAO	3.0	kg de racao por litro produzido acima da base
MAXIMO_POR_REFEICAO	6.0	kg maximo de racao por refeicao
CACHE_TTL_SEGUNDOS	60	Intervalo de recarga dos dados do Firebase

Documento gerado em Maio de 2026 para o projeto MilkShow.

Toda a documentacao reflete o estado atual do codigo em producao.